

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS
GERAIS**

Câmpus Poços de Caldas – Ciência da Computação

PROJETO SALVOU – CRM

Definição da Arquitetura do Sistema: Sprint 3

Gabriel Alencar do Lago
Daniley Fernanda Poscidonio
Yasmin Alcisio Corona
Henrique de Souza Rocha
Brenda Tristão
João Gabriel Bernardes

Sumário

1	Relatório Técnico de Engenharia de Software	2
1.1	Descrição e Justificativa da Arquitetura	2
1.1.1	Justificativas Técnicas	2
1.2	Organização do Sistema e Responsabilidades	2
1.2.1	Exemplo de Lógica de Domínio (<code>Cliente.java</code>)	2
1.2.2	Exemplo de Rota de Controle (<code>ClienteController.java</code>)	3
1.3	Comunicação entre Componentes e Padrão REST	3
1.3.1	Comunicação Interna (Injeção de Dependência)	3
1.3.2	Comunicação Externa (Híbrida: MVC e REST)	3
2	Explicação da Arquitetura para Pessoas Leigas	4
2.1	O Recepcionista (A Camada Controller)	4
2.2	O Gerente de Contas (A Camada Model / Entity)	4
2.3	O Arquivista do Cofre (A Camada Repository)	4
2.4	Vantagem Estratégica da Organização	5

1 Relatório Técnico de Engenharia de Software

1.1 Descrição e Justificativa da Arquitetura

O sistema **Salvou** adota a **Arquitetura em Camadas (Layered Architecture)** acoplada ao padrão de inversão de controle fornecido pelo ecossistema Spring Boot. A separação clara entre as regras de representação visual, lógica de negócios e persistência de dados garante que o sistema seja altamente coeso e fracamente acoplado.

1.1.1 Justificativas Técnicas

- **Abstração da Camada de Dados (JPA/Hibernate):** Toda a comunicação com o banco de dados relacional é feita via mapeamento objeto-relacional (ORM). Isso significa que as regras de persistência se apoiam em interfaces orientadas a objetos, isolando a aplicação do dialeto SQL nativo.
- **Independência de Camadas:** Alterações na estrutura visual dos arquivos HTML localizados na pasta `templates` não impactam as entidades de negócio (`Cliente`) ou as consultas de persistência (`ClienteRepository`).

1.2 Organização do Sistema e Responsabilidades

Tabela 1.1: Responsabilidade Primária por Componente

Camada / Componente	Responsabilidade Primária
Model / Entity (<code>Cliente.java</code>)	Mapear os atributos relacionais e regras intrínsecas ao domínio.
Repository (<code>ClienteRepository.java</code>)	Abstrair operações de persistência e CRUD via JPA.
Controller (<code>ClienteController.java</code>)	Interceptar rotas HTTP e gerenciar o fluxo de dados do sistema.

1.2.1 Exemplo de Lógica de Domínio (`Cliente.java`)

O método abaixo encapsula a inteligência de somar o valor de todas as vendas daquele cliente utilizando a API de Streams do Java:

```
1 public double getValorTotalVendas() {
2     if (this.vendas == null || this.vendas.isEmpty()) {
3         return 0.0;
4     }
5     return this.vendas.stream()
6         .mapToDouble(v -> v.getValorTotal() != null ? v.
7             getValorTotal() : 0.0)
8         .sum();
9 }
```

1.2.2 Exemplo de Rota de Controle (ClienteController.java)

O método intercepta a requisição de cadastro enviada pela interface gráfica e invoca o repositório seguro:

```
1 @PostMapping("/salvar")
2 public String salvar(Cliente cliente, RedirectAttributes
   attributes) {
3     if (repository.existsByCpf(cliente.getCpf())) {
4         attributes.addFlashAttribute("mensagemErro", "Este CPF
j     est cadastrado!");
5         return "redirect:/novo";
6     }
7     repository.save(cliente);
8     return "redirect:/clientes";
9 }
```

1.3 Comunicação entre Componentes e Padrão REST

1.3.1 Comunicação Interna (Injeção de Dependência)

As camadas se comunicam através de desacoplamento gerenciado pelo Spring Container. Utilizando a anotação `@Autowired`, os controladores obtêm instâncias ativas dos repositórios sem a necessidade de acoplamento rígido via operador `new`.

1.3.2 Comunicação Externa (Híbrida: MVC e REST)

O sistema opera utilizando requisições síncronas para carregar as telas através do Thymeleaf e, concorrentemente, expõe rotas assíncronas baseadas nas anotações `@ResponseBody` e `@RequestBody` para comunicação direta via objetos textuais estruturados em JSON (padrão REST).

2 Explicação da Arquitetura para Pessoas Leigas

Para entender como o sistema **Salvou** funciona por dentro sem precisar entender de linhas de código complexas, imagine que o programa é como um **Escritório de Contabilidade Automatizado e Altamente Organizado**.

Para que as informações da empresa não fiquem espalhadas ou bagunçadas, o escritório divide o trabalho rigidamente entre três funcionários principais:

2.1 O Recepcionista (A Camada Controller)

Quando um cliente chega no escritório ou liga pedindo uma informação, ele não entra direto nas salas privadas dos fundos; ele é atendido pelo **Recepcionista**.

- **No Salvou:** O arquivo `ClienteController` é esse funcionário. Quando você clica no botão “Salvar Cliente” na tela, quem recebe esse comando é ele. Ele confere se os papéis básicos estão preenchidos de forma coerente. Se você digitar um CPF que já existe, o Recepcionista barra na hora e diz: *“Ei, esse documento já está na nossa lista, preencha o formulário de novo!”*.

2.2 O Gerente de Contas (A Camada Model / Entity)

Depois que o Recepcionista confere e aceita os papéis iniciais, os dados passam a pertencer ao **Gerente de Contas**, que é quem domina as regras de funcionamento do negócio.

- **No Salvou:** O arquivo `Cliente.java` faz esse papel. Ele armazena o nome, telefone, endereço e e-mail de cada cliente. Ele também é inteligente: se a diretoria do escritório perguntar quanto o Cliente X já gastou na empresa até hoje, o Gerente usa uma fórmula matemática própria para somar todas as notas fiscais de venda daquele cliente e entregar a resposta exata em milissegundos.

2.3 O Arquivista do Cofre (A Camada Repository)

O Gerente de Contas é muito inteligente para fazer cálculos rápidos, mas ele não armazena caixas pesadas de relatórios embaixo da sua própria mesa. Quando ele precisa guardar uma ficha permanentemente ou consultar históricos antigos, ele faz um pedido para o **Arquivista**.

- **No Salvou:** O arquivo `ClienteRepository` é o arquivista que cuida do cofre de dados. Ele tem superpoderes para vasculhar gavetas de arquivos gigantescas instantaneamente e trazer a informação limpa.

2.4 Vantagem Estratégica da Organização

Se o escritório colocasse o Recepcionista para atender o telefone, calcular impostos e correr até o cofre de arquivos ao mesmo tempo, o serviço viraria um caos completo.

No **Salvou**, tudo é separado. Se amanhã o escritório crescer e decidir trocar o cofre de arquivos em arquivo por um servidor de banco de dados corporativo imenso, nós só precisaremos alterar o treinamento do **Arquivista (Repository)**. O Recepcionista da entrada (**Controller**) e as regras matemáticas do gerente (**Model**) continuarão trabalhando exatamente do mesmo jeito, sem perceber a mudança e sem interromper o atendimento.